

به نام خدای رنگین کمان

گزارش فاز 2 پروژه اصول طراحی کامپایلر (تحلیلگر نحوی)

مریم خوش فکر 990122680008

فرید افراخته 990122680092

```
phase2_parser.cup - Notepad
File Edit Format View Help
non terminal expression_statement, statement_expression, assignment, assignment_operator, do_statement;

start with goal;

// The Syntactic Grammar
goal ::=      compilation_unit
      ;

compilation_unit ::= func_declaration
      ;

func_declaration ::=
      func_header func_body
      ;

func_header ::=
      VOID func_declarator
      ;

func_declarator ::=
      IDENTIFIER LPAREN RPAREN
      ;

func_body ::=  block
      |
      SEMICOLON
      ;

block ::=      LBRACE block_statements_opt RBRACE
      ;

block_statements_opt ::=
      |
      block_statements
      ;

block_statements ::=
      statement
      |
      block_statements statement
      ;
```

در قسمت پارسر شروع با **goal** است که به **compilation_unit** میرود که طبق قواعد برای زبان با **function** شروع می شود که تابع عم از دو قسمت بدنه و هدر تشکیل شده است برای هداول باید با کلمه **Void** شروع میشود و بعد از اسم تابع یا توکن شناساگر گرفته شده از اسکنر و با پرانتز باز و بسته قسمت هدر تمام میشود.

برای بدنه هم میتواند به **block** یا ترمینال **semicolon** برود که این انتخاب برای این است که فقط تابع اعلان یا تعریف شود اگر تعریف شود به **block** رفته که با ترمینال های {} شروع و تمام میشود و بین آن دو ترمینال ,یر ترمینال **block_statement_opt** وجود دارد که اجازه دهد تابع خالی باشد یا دارای چند **statement** باشد که در این صورت به **block_statement** می رود.

```
phase2_parser.cup - Notepad
File Edit Format View Help
block_statements ::=
    statement
    | block_statements statement
;

statement ::=
    statement2
    | if_statement
    | while_statement
    | for_statement
;

statement2 ::=
    block
    | empty_statement
    | expression_statement
    | do_statement
;

expression_statement ::=
    statement_expression SEMICOLON
;
statement_expression ::=
    assignment
;

empty_statement ::=
    SEMICOLON
;

assignment ::= IDENTIFIER assignment_operator INTEGER
;

assignment_operator ::=
    EQ
;
<
```

بعد برای هر **statement** میتوان **if** یا **while** یا **for** یا به **statement 2** رفت و دوباره به **block** جدید یا به ترمینال رفته تا یک مقدار دهی به متغیر دهد یا یک **statement** خالی فقط یا ترمینال ; پارس کرد.

گرامر **expression_statement** برای همان مقدار دهی به متغیر است که به **statement_expression** با ترمینال ; میرود و بعد **assignment** که از شناساگر عملگر ها و در انتها از یک عدد صحیح تشکیل شده است.

```

phase2_parser.cup - Notepad
File Edit Format View Help
EQ
;

if_statement ::=
    IF LPAREN expression RPAREN statement
;

expression_opt ::=
    | expression
;
expression ::= assignment_expression
;

assignment_expression ::=
    conditional_expression
;

conditional_expression ::=
    relational_expression
;

relational_expression ::=
    IDENTIFIER LT IDENTIFIER
    | IDENTIFIER GT IDENTIFIER
;

while_statement ::=
    WHILE LPAREN expression RPAREN statement
;
do_statement ::=
    DO statement WHILE LPAREN expression RPAREN SEMICOLON
;

for_statement ::=
    <

```

برای دستور شرطی **if** هم که از **statement** بالا به این قاعده می آید از ترمینال **if** پرانتز باز **expression** پرانتز بسته و در نهایت **statement** تشکیل شده است **expression** هم در نهایت به عبارات شرطی یعنی **Conditional _ expression** میرود که برای این پروژه عبارات شرطی بزرگتر و کوچکتر استفاده شده است که در **relational_expression** قرار دارد.

گرامر **while** هم از ترمینال **while** پرانتز باز **expression** مانند **if** بالا و پرانتز بسته و در نهایت **statement** تشکیل شده که بتواند در داخل **while** حلقه ها با مقداری های دیگر قرار داد. و در نهایت گرامر **for** که از ترمینال **for** پرانتز باز **semicolon** و **expression_opt** برای این است که می توان **expression** بین دو ; قرار نداد یا در نهایت به **expression** رفت و عبارت شرطی قرار داد.

scanner_phase2.flex - Notepad

File Edit Format View Help

```

"for"          { return symbol(FOR); }
"if"           { return symbol(IF); }
"void"         { return symbol(VOID); }
"while"        { return symbol(WHILE); }
"do"           { return symbol(DO); }

/* null literal */
"null"         { return symbol(NULL_LITERAL); }
/* separators */
"("            { return symbol(LPAREN); }
")"            { return symbol(RPAREN); }
"{"            { return symbol(LBRACE); }
"}"            { return symbol(RBRACE); }

";"           { return symbol(SEMICOLON); }

/* operators */
"="            { return symbol(EQ); }
">"            { return symbol(GT); }
"<"            { return symbol(LT); }

/* string literal */
\"            { yybegin(STRING); string.setLength(0); }

/* character literal */
\'            { yybegin(CHARLITERAL); }

/* numeric literals */

/* This is matched together with the minus, because the number is too big to
   be represented by a positive integer. */
{DecIntegerLiteral} { return symbol(INTEGER, Integer.valueOf(yytext())); }

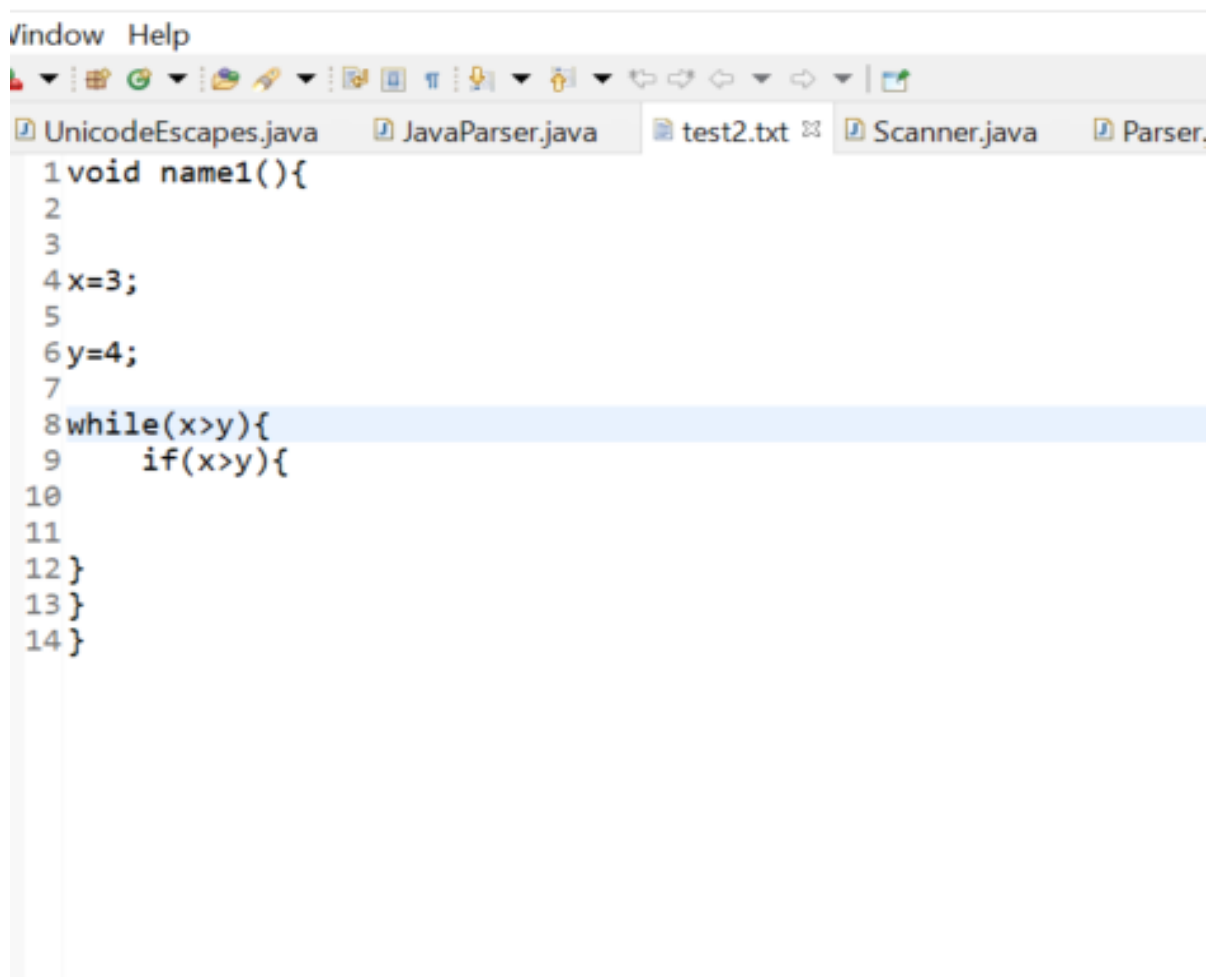
/* comments */
{Comment}         { /* ignore */ }

/* whitespace */
{WhiteSpace}      { /* ignore */ }

```

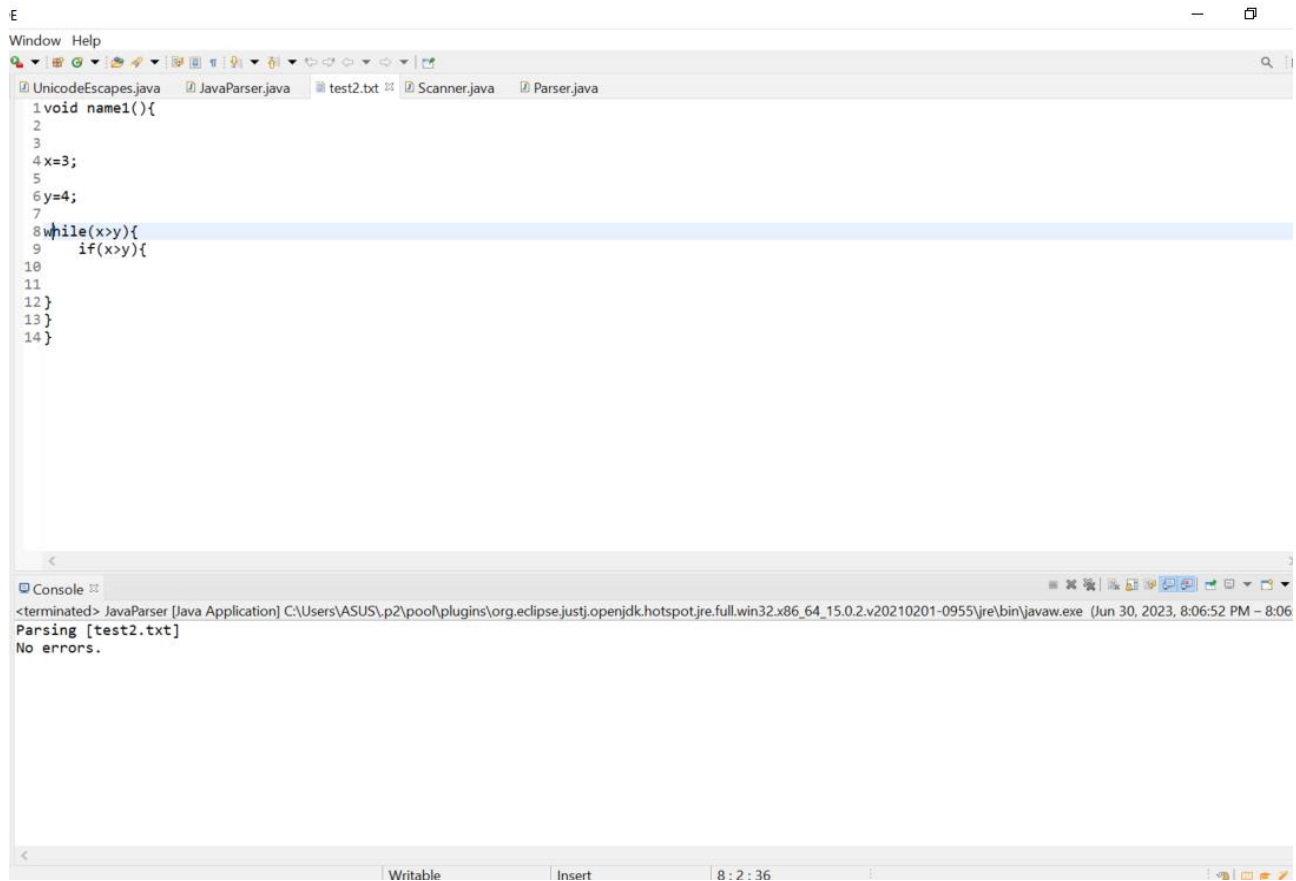
<

برای اسکنر مجبور شدیم اسکنر جاوا را تغییر دهیم و برای همین مسئله اسکنر خود جاوا را مطابق گرامری که برای فاز دوم ساخته شده تغییر دادیم و خیلی از کلمات کلیدی و عملگرها حذف شدند .



```
Window Help
UnicodeEscapes.java  JavaParser.java  test2.txt  Scanner.java  Parser.
1 void name1(){
2
3
4 x=3;
5
6 y=4;
7
8 while(x>y){
9     if(x>y){
10
11
12 }
13 }
14 }
```

در تصویر بالا کد تست را مشاهده میکنیم .



تصویر بالا خروجی کد را روی تست نشان مدهد که همانطور که ملاحظه میکنید بدون ارور است.

با ایجاد یک تغییر مثل حذف **w** در **while** برنامه از کد ما خطا میگیرد که در تصویر زیر قابل مشاهده است.

```
1 void name1(){
2
3
4 x=3;
5
6 y=4;
7
8 while(x>y){
9     if(x>y){
10
11
12 }
13 }
14 }
```

Console

terminated> JavaParser [Java Application] C:\Users\ASUS\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_15.0.2.v20210201-0955\jre\bin\javaw.exe (Jun 30, 2023, 8:10:51 PM - 8:10:51 PM)

Parsing [test2.txt]

Error in line 8, column 5 : Syntax error
instead expected token classes are []

Error in line 8, column 5 : Couldn't repair and continue parse

java.lang.RuntimeException: Fatal Syntax Error

at parser3.Parser.report_fatal_error(Parser.java:218)
at java_cup.runtime.lr_parser.unrecovered_syntax_error(lr_parser.java:539)
at java_cup.runtime.lr_parser.parse(lr_parser.java:731)
at parser3.JavaParser.main(JavaParser.java:26)

مشخص شدن خطای سینتکس در خط 6 توسط کنسول.